Support Vector Machine

Finding the Maximum-Margin Hyperplane

Dr. Nikhilanand Arya

Indian Institute of Information Technology Allahabad

From Data Points to the Maximum-Margin Decision Boundary

Maximum Margin Classifier

The Problem

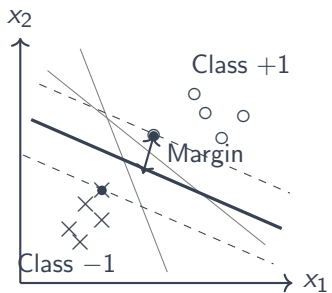
For linearly separable data, we can draw **many different separating hyperplanes**.

Key Question

Which one is the **best**?

Maximum Margin Idea

Choose the hyperplane that has the **largest possible distance** from the nearest training points.

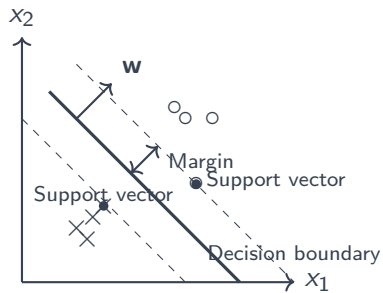


Maximum Margin Classifier

Answer

The best hyperplane is the one that:

- Maximizes the distance to the nearest points
- Maintains a balanced margin for both classes
- Separates the two classes correctly



Final Insight

Maximum Margin

=

Maximum distance to the closest points

Mathematical Formulation of SVM

1. Decision Hyperplane

The separating hyperplane is defined as

$$\boxed{w^T x + b = 0}$$

where

- w = weight vector, **normal (perpendicular)** to the hyperplane
- x = input feature vector
- b = bias, which shifts the hyperplane

2. What Is the Margin?

The **margin** γ is the perpendicular distance from the decision hyperplane to the closest training point.

Mathematical Formulation of SVM

3. Why Does **w** Matter?

- The shortest distance to a hyperplane is always **perpendicular**.
- The vector perpendicular to $w^T x + b = 0$ is w.
- Therefore, the margin is measured **along the direction of** w.

Margin = Perpendicular distance to the closest point

Distance to Hyperplane: Projection Idea

Step 1: Start with a Point

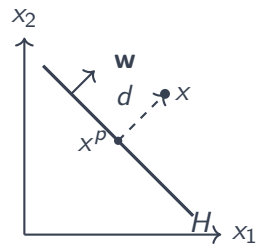
Let x be a point outside the hyperplane H.
We want the **shortest distance** from x to H.

Step 2: Projection Point

Let x^p be the projection of x onto the hyperplane.
Then the distance vector is

$$\boxed{d = x - x^p}$$

$$\boxed{x^p = x - d}$$



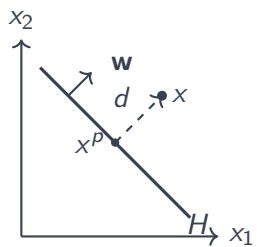
Distance to Hyperplane: Projection Idea

Why **w**?

The shortest path to a hyperplane is perpendicular to the hyperplane.
Since **w** is normal to the hyperplane,

$$\boxed{d = \alpha \mathbf{w}}$$

for some scalar α.



Distance to Hyperplane: From Projection to Distance

Step 3: Substitute in the Hyperplane

Since x^p lies on H,

$$w^T x^p + b = 0$$

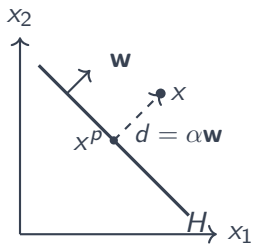
Using

$$x^p = x - d, \qquad d = \alpha w,$$

$$w^T(x - \alpha w) + b = 0$$

$$w^T x - \alpha w^T w + b = 0$$

$$\boxed{\alpha = \frac{w^T x + b}{w^T w}}$$



Distance to Hyperplane: From Projection to Distance

Step 4: Calculate the Distance

Since $d = \alpha w$,

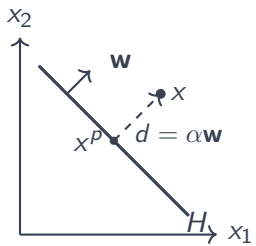
$$\|d\| = \sqrt{d^T d} = \sqrt{\alpha^2 w^T w} = |\alpha| \sqrt{w^T w}$$

Since

$$w^T w = \|w\|^2,$$

we obtain

$$\boxed{\|d\| = \frac{|w^T x + b|}{\|w\|}}$$



Key Idea

Shortest distance is measured **along the normal w**.

The Distance Formula Gives Us the Margin

For any point x and hyperplane

$$w^T x + b = 0,$$

the perpendicular distance is

$$\boxed{d(x, H) = \frac{|w^T x + b|}{\|w\|}}$$

We have two classes. Instead of asking:

"How do I separate the classes?"

ask:

"How far apart can I keep them?"

Margin of Hyperplane

Margin Definition

The **margin** of a hyperplane H with respect to dataset D is the minimum perpendicular distance between the hyperplane and any training point:

$$\gamma(w, b) = \min_{x \in D} \frac{|w^T x + b|}{\|w\|}$$

Margin of Hyperplane

What Does This Mean?

- The hyperplane is:

$$w^T x + b = 0$$

- For each training point x, compute its perpendicular distance:

$$d(x, H) = \frac{|w^T x + b|}{\|w\|}$$

- The **margin** is the distance to the **closest training point**.

Key Idea

The margin is controlled by the closest points to the decision boundary.

Why Maximum Margin?

Key Observation

A good separating hyperplane should stay as far as possible from **both classes**.

- It lies approximately in the **middle between the two classes**.
- The closest points from both classes are **equally distant**.

Critical Idea

The margin is determined by the **closest training points**:

$$\boxed{\gamma = \text{distance to the closest point from either class}}$$

Why Maximum Margin?

Canonical Margin Boundaries

For SVM, we scale the hyperplane so that the two closest boundaries are:

$$\boxed{w^T x + b = +1} \quad \text{and} \quad \boxed{w^T x + b = -1}$$

The decision boundary lies exactly between them:

$$\boxed{w^T x + b = 0}$$

Next Step

What is the distance between these two boundaries?

Maximum Margin as an Optimization Problem

Goal

Find the separating hyperplane that places the two classes **as far apart as possible**:

$$\max_{w,b} \ \gamma(w, b)$$

where γ is the margin.

Separation Constraint

For every training point (x_i, y_i):

$$y_i(w^T x_i + b) \geq 0$$

This ensures that every point is correctly classified.

Maximum Margin as an Optimization Problem

What Does the Constraint Mean?

- $y_i = +1$:

$$w^T x_i + b \geq 0$$

- $y_i = -1$:

$$w^T x_i + b \leq 0$$

But Is This Enough?

No. Many separating hyperplanes satisfy this condition.

We need an additional constraint to identify the **maximum-margin hyperplane**.

Rewriting the Margin Objective

Margin Expression

For a hyperplane

$$w^T x + b = 0,$$

the margin is the distance to the closest training point:

$$\boxed{\gamma(w, b) = \min_i \frac{|w^T x_i + b|}{\|w\|}}$$

maximum-margin learning can be written as:

$$\boxed{\max_{w,b} \frac{1}{\|w\|} \min_i |w^T x_i + b|}$$

Rewriting the Margin Objective

Key Insight

The margin contains two components:

$$\underbrace{|w^T x_i + b|}_{\text{signed distance before normalization}} \qquad \underbrace{\|w\|}_{\text{normalization}}$$

We want a large separation with a small $\|w\|$.

Next Step

To remove the absolute value, SVM uses the labels $y_i \in \{-1, +1\}$ and chooses the canonical scaling:

$$\boxed{y_i(w^T x_i + b) \geq 1}$$

Scale Invariance of a Hyperplane

Why Can We Fix the Scale?

The hyperplane

$$w^T x + b = 0$$

does not change if both w and b are multiplied by the same non-zero constant λ:

$$\boxed{(w, b) \equiv (\lambda w, \lambda b)}$$

because

$$w^T x + b = 0 \iff \lambda(w^T x + b) = 0.$$

Scale Invariance of a Hyperplane

Fixing the Scale

We choose the scale such that the closest training points satisfy

$$\boxed{|w^T x_i + b| = 1}$$

or, using the class labels $y_i \in \{-1, +1\}$,

$$\boxed{y_i(w^T x_i + b) \geq 1.}$$

Scale Invariance of a Hyperplane

Resulting Margin

The distance from the decision boundary to the closest point is

$$\boxed{\gamma_{\text{one-sided}} = \frac{1}{\|w\|}}$$

Therefore, the complete margin between the two boundaries is

$$\boxed{\gamma_{\text{full}} = \frac{2}{\|w\|}}$$

Final SVM Optimization Problem

From Maximum Margin to Optimization

The full margin is

$$\gamma = \frac{2}{\|w\|}.$$

Therefore, maximizing the margin is equivalent to minimizing $\|w\|$:

$$\boxed{\max_{w,b} \frac{2}{\|w\|} \quad \Longleftrightarrow \quad \min_{w,b} \frac{1}{2}\|w\|^2}$$

Final SVM Optimization Problem

Final SVM Formulation

$$\boxed{\min_{w,b} \frac{1}{2}\|w\|^2}$$

subject to

$$\boxed{y_i(w^T x_i + b) \geq 1, \qquad \forall i}$$

What Is SVM Trying to Do?

Find the separating hyperplane with the largest possible margin.

$$\text{Large margin} \iff \text{small } \|w\|^2$$

From Constrained to Lagrangian Optimization

Primal Problem

The Hard-Margin SVM solves:

$$\min_{w,b} \frac{1}{2} w^T w$$

subject to

$$y_i(w^T x_i + b) \geq 1, \qquad \forall i$$

Key Idea: Lagrange Multipliers

Introduce a non-negative multiplier $\alpha_i \geq 0$ for each constraint.
The constraints can be written as:

$$y_i(w^T x_i + b) - 1 \geq 0.$$

Lagrangian Formulation

Lagrangian Function

Starting from the SVM constraint

$$y_i(w^T x_i + b) \geq 1,$$

write it as

$$y_i(w^T x_i + b) - 1 \geq 0.$$

The Lagrangian is:

$$\boxed{L(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{n} \alpha_i \left[y_i(w^T x_i + b) - 1 \right]}$$

Lagrangian Formulation

Lagrange Multipliers

Each constraint has a non-negative multiplier:

$$\boxed{\alpha_i \geq 0, \qquad \forall i}$$

What Does α_i Tell Us?

- $\alpha_i = 0$: training point does not influence the final boundary.
- $\alpha_i > 0$: training point influences the boundary.
- These influential points are called **support vectors**.

Optimality Condition: Finding w

Differentiate the Lagrangian

At the optimum, the derivative with respect to w must be zero:

$$\frac{\partial L}{\partial w} = 0$$

Therefore,

$$w - \sum_{i=1}^{n} \alpha_i y_i x_i = 0$$

and hence

$$\boxed{w = \sum_{i=1}^{n} \alpha_i y_i x_i}$$

Optimality Condition: Finding w

Key Interpretation

The weight vector w is a **linear combination of the training points**:

$$w = \alpha_1 y_1 x_1 + \alpha_2 y_2 x_2 + \cdots + \alpha_n y_n x_n.$$

Why Are They Called Support Vectors?

- If $\alpha_i = 0$, point x_i contributes nothing to w.
- If $\alpha_i > 0$, point x_i contributes to the decision boundary.
- These points are called **Support Vectors**.

The decision boundary is supported by only a subset of the data.

Bias Constraint from Optimality

Derivative with Respect to b

Starting from the Lagrangian:

$$L = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{n} \alpha_i \left[y_i(w^T x_i + b) - 1 \right]$$

$$\frac{\partial L}{\partial b} = 0$$

$$-\sum_{i=1}^{n} \alpha_i y_i = 0$$

$$\boxed{\sum_{i=1}^{n} \alpha_i y_i = 0}$$

Bias Constraint from Optimality

What Does This Constraint Mean?

The weighted contribution of the two classes must balance:

$$\sum_{y_i=+1} \alpha_i = \sum_{y_i=-1} \alpha_i$$

Key Takeaway

$$\boxed{\sum_i \alpha_i y_i = 0}$$

is one of the constraints that appears in the **SVM dual optimization problem**.

From Primal to Dual

Recall the Optimal Weight Vector

From the optimality condition:

$$\boxed{w = \sum_{i=1}^{n} \alpha_i y_i x_i}$$

From Primal to Dual

The primal objective contains:

$$\frac{1}{2}\|w\|^2 = \frac{1}{2}w^T w.$$

Substituting w:

$$\frac{1}{2}\left(\sum_i \alpha_i y_i x_i\right)^T \left(\sum_j \alpha_j y_j x_j\right).$$

Expand the Product

$$\boxed{\frac{1}{2}\|w\|^2 = \frac{1}{2}\sum_i \sum_j \alpha_i \alpha_j y_i y_j x_i^T x_j}$$

Expanding the Dual Objective

Two Terms Appear

After substituting

$$w = \sum_i \alpha_i y_i x_i$$

into the Lagrangian, we obtain two main terms.

Quadratic Term

$$\boxed{\frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j (x_i^T x_j)}$$

Expanding the Dual Objective

Two Terms Appear

After substituting

$$w = \sum_i \alpha_i y_i x_i$$

into the Lagrangian, we obtain two main terms.

Linear Term

$$\boxed{\sum_{i=1}^{n} \alpha_i}$$

Expanding the Dual Objective

Two Terms Appear

After substituting

$$w = \sum_i \alpha_i y_i x_i$$

into the Lagrangian, we obtain two main terms.

Dual Objective

Combining both terms:

$$\boxed{W(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j (x_i^T x_j)}$$

Key Insight: Dot Product Dependence

Critical Observation

The SVM dual objective depends on the training data only through pairwise inner products:

$$\boxed{x_i^T x_j}$$

What Does This Mean?

- We do not need to explicitly manipulate the feature vectors in the dual objective.
- We only need to compute the similarity between pairs of training points:

$$x_i^T x_j.$$

Key Insight: Dot Product Dependence

Why Is This Powerful?

If we can replace the inner product with a function

$$\boxed{K(x_i, x_j)}$$

we can work with data in a transformed feature space **without explicitly computing that transformation**.

This Enables the Kernel Trick

Non-linear decision boundaries in high-dimensional feature spaces

KKT Conditions in SVM

Lagrangian

For the hard-margin SVM,

$$L(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_i \alpha_i \left[y_i(w^T x_i + b) - 1 \right], \qquad \alpha_i \geq 0$$

At the optimal solution, the following conditions must hold:

$$\begin{aligned} y_i(w^T x_i + b) - 1 &\geq 0 \quad \text{(Primal feasibility)} \\ \alpha_i &\geq 0 \quad \text{(Dual feasibility)} \\ \alpha_i \left[y_i(w^T x_i + b) - 1 \right] &= 0 \quad \text{(Complementary slackness)} \end{aligned}$$

KKT Conditions in SVM

Key SVM Insight

$$\boxed{\alpha_i > 0 \quad \Longrightarrow \quad y_i(w^T x_i + b) = 1}$$

Only these points lie exactly on the margin boundaries.

$$\therefore \quad \boxed{\text{They are the Support Vectors}}$$

KKT: Stationarity

1. Derivative with respect to w

Starting from

$$L(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_i \alpha_i \left[y_i(w^T x_i + b) - 1 \right],$$

stationarity requires

$$\frac{\partial L}{\partial w} = 0.$$

Therefore,

$$\boxed{w = \sum_i \alpha_i y_i x_i}$$

KKT: Stationarity

2. Derivative with respect to b

Similarly,

$$\frac{\partial L}{\partial b} = 0$$

gives

$$\boxed{\sum_i \alpha_i y_i = 0}$$

KKT: Stationarity

Geometric Meaning

$$\boxed{w = \sum_i \alpha_i y_i x_i}$$

The optimal normal vector w is a **weighted combination of training points**. Only points with $\alpha_i > 0$ contribute to w.

KKT: Feasibility Conditions

1. Primal Feasibility

Every training point must satisfy the SVM margin constraint:

$$\boxed{y_i(w^T x_i + b) - 1 \geq 0}$$

Equivalently,

$$\boxed{y_i(w^T x_i + b) \geq 1}$$

2. Dual Feasibility

The Lagrange multipliers must satisfy

$$\boxed{\alpha_i \geq 0}$$

KKT: Feasibility Conditions

Interpretation

- **Primal:** Every training point must lie on the correct side of the margin.
- **Dual:** Each Lagrange multiplier is **non-negative**.

Primal feasibility + Dual feasibility

KKT: Complementary Slackness

Complementary Slackness Condition

For every training point,

$$\boxed{\alpha_i \left[y_i(w^T x_i + b) - 1 \right] = 0}$$

Critical Insight

The product can be zero in two ways:

$$\boxed{\alpha_i > 0 \implies y_i(w^T x_i + b) = 1}$$

$$\boxed{\alpha_i = 0 \implies y_i(w^T x_i + b) > 1}$$

KKT: Complementary Slackness

Geometric Interpretation

- $\alpha_i > 0 \Rightarrow$ point lies **exactly on the margin**
- $\alpha_i = 0 \Rightarrow$ point lies **away from the margin**

$$\boxed{\alpha_i > 0 \quad \Longrightarrow \quad \text{Support Vector}}$$

Finding Bias b

Step 1: Use the KKT Condition

For a support vector, the margin constraint is active:

$$y_i(w^T x_i + b) = 1$$

Step 2: Solve for b

Since $y_i \in \{-1, +1\}$ and $y_i^2 = 1$,

$$y_i w^T x_i + y_i b = 1$$

Multiplying by y_i:

$$\boxed{b = y_i - w^T x_i}$$

Finding Bias b

How to Compute b

Choose any support vector (x_i, y_i):

$$\boxed{b = y_i - w^T x_i}$$

If several support vectors are available, compute

$$\boxed{b = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \left(y_i - w^T x_i \right)}$$

Key Idea

The support vectors determine both the **orientation** of the hyperplane (w) and its **position** (b).

Numerical Example of SVM

Dataset

Class (+1):

$$(3, 1), (3, -1), (6, 1), (6, -1)$$

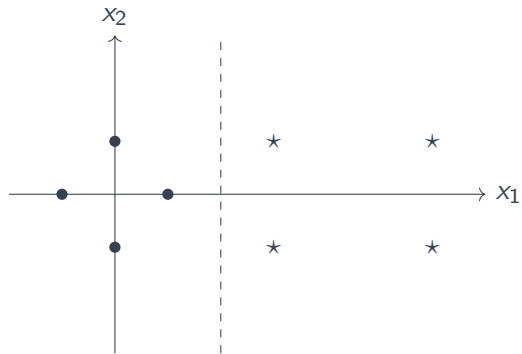
Class (-1):

$$(1, 0), (0, 1), (0, -1), (-1, 0)$$

Goal

Find the optimal separating hyperplane using SVM

Data Visualization



Observation

Hyperplane is vertical (separates at $x_1 \approx 2$)

Support Vectors Selection

Closest Points

Choose points nearest to boundary:

- $S_1 = (1, 0)$ (class -1)
- $S_2 = (3, 1)$ (class $+1$)
- $S_3 = (3, -1)$ (class $+1$)

Why These?

- They lie closest to separating boundary
- Define the margin

Augmented Representation

Add Bias Term

$$\bar{x} = (x_1, x_2, 1)$$

Support Vectors

$$\bar{S}_1 = (1, 0, 1), \quad y_1 = -1$$
$$\bar{S}_2 = (3, 1, 1), \quad y_2 = +1$$
$$\bar{S}_3 = (3, -1, 1), \quad y_3 = +1$$

SVM: Support-Vector Equations

Start from the KKT equality

For every support vector S_i,

$$y_i \left(\mathbf{w}^T S_i + b \right) = 1$$

Since $y_i \in \{-1, +1\}$,

$$\boxed{\mathbf{w}^T S_i + b = y_i}$$

SVM: Support-Vector Equations

Substitute the weight vector

Recall,

$$\mathbf{w} = \sum_j \alpha_j y_j S_j$$

Therefore,

$$\boxed{\sum_j \alpha_j y_j \left(S_j^T S_i \right) + b = y_i}$$

Important

This is the correct support-vector equation when b **is not ignored**.

Three Support Vectors: Four Unknowns

Assume

$$y_1 = -1, \qquad y_2 = +1, \qquad y_3 = +1.$$

Support-vector equations

For S_1:

$$-\alpha_1(S_1^T S_1) + \alpha_2(S_2^T S_1) + \alpha_3(S_3^T S_1) + b = -1$$

For S_2:

$$-\alpha_1(S_1^T S_2) + \alpha_2(S_2^T S_2) + \alpha_3(S_3^T S_2) + b = +1$$

For S_3:

$$-\alpha_1(S_1^T S_3) + \alpha_2(S_2^T S_3) + \alpha_3(S_3^T S_3) + b = +1$$

Three Support Vectors: Four Unknowns

KKT condition from b

$$-\alpha_1 + \alpha_2 + \alpha_3 = 0$$

Unknowns: $\alpha_1, \alpha_2, \alpha_3, b$

Augmented-vector trick

$$\boxed{\tilde{S}_i = \begin{bmatrix} S_i \\ 1 \end{bmatrix}, \qquad \tilde{\mathbf{w}} = \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}}$$

gives

$$\boxed{\tilde{\mathbf{w}}^T \tilde{S}_i = \mathbf{w}^T S_i + b}$$

So b can be absorbed into the algebra using an **augmented vector**.

For every augmented support vector $\tilde{S}_i$

$$y_i \left(\tilde{\mathbf{w}}^T \tilde{S}_i \right) = 1$$

$$\boxed{\tilde{\mathbf{w}}^T \tilde{S}_i = y_i}$$

Three Support Vectors: Four Unknowns

$$y_1 = -1, \qquad y_2 = +1, \qquad y_3 = +1.$$

Augmented Support-vector equations

For $\tilde{S}_1$:

$$-\alpha_1(\tilde{S}_1^T \tilde{S}_1) + \alpha_2(\tilde{S}_2^T \tilde{S}_1) + \alpha_3(\tilde{S}_3^T \tilde{S}_1) = -1$$

For $\tilde{S}_2$:

$$-\alpha_1(\tilde{S}_1^T \tilde{S}_2) + \alpha_2(\tilde{S}_2^T \tilde{S}_2) + \alpha_3(\tilde{S}_3^T \tilde{S}_2) = +1$$

For $\tilde{S}_3$:

$$-\alpha_1(\tilde{S}_1^T \tilde{S}_3) + \alpha_2(\tilde{S}_2^T \tilde{S}_3) + \alpha_3(\tilde{S}_3^T \tilde{S}_3) = +1$$

Why Augmented Form is Useful

Without augmentation:

$$w^T x + b$$

With augmentation:

$$w'^T x' \quad \text{where } x' = (x, 1)$$

- Eliminates separate handling of b
- Simplifies optimization
- Cleaner matrix formulation
- Useful in dual derivation

Insight

Bias becomes part of the weight vector

Augmented Formulation (Including Bias)

Augmented Representation

$$\bar{S}_i = (x_1, x_2, 1)$$

Support Vectors

$$\bar{S}_1 = (1, 0, 1), \quad y_1 = -1$$
$$\bar{S}_2 = (3, 1, 1), \quad y_2 = +1$$
$$\bar{S}_3 = (3, -1, 1), \quad y_3 = +1$$

Model:

$$w' = \sum \alpha_i y_i \bar{S}_i$$

KKT Conditions (Augmented Form)

Equality Condition

$$\sum_j \alpha_j y_j (\bar{S}_j \cdot \bar{S}_i) = y_i$$

$$\bar{S}_1 \cdot \bar{S}_1 = 2$$
$$\bar{S}_2 \cdot \bar{S}_1 = 4$$
$$\bar{S}_3 \cdot \bar{S}_1 = 4$$
$$\bar{S}_2 \cdot \bar{S}_2 = 11$$
$$\bar{S}_3 \cdot \bar{S}_2 = 9$$
$$\bar{S}_3 \cdot \bar{S}_3 = 11$$

System of Equations

For $i = 1$:

$$\alpha_1(-1)(2) + \alpha_2(1)(4) + \alpha_3(1)(4) = -1$$

$$-2\alpha_1 + 4\alpha_2 + 4\alpha_3 = -1$$

For $i = 2$:

$$\alpha_1(-1)(4) + \alpha_2(1)(11) + \alpha_3(1)(9) = +1$$

$$-4\alpha_1 + 11\alpha_2 + 9\alpha_3 = 1$$

For $i = 3$:

$$\alpha_1(-1)(4) + \alpha_2(1)(9) + \alpha_3(1)(11) = +1$$

$$-4\alpha_1 + 9\alpha_2 + 11\alpha_3 = 1$$

Solution for α_i

System

$$-2\alpha_1 + 4\alpha_2 + 4\alpha_3 = -1$$
$$-4\alpha_1 + 11\alpha_2 + 9\alpha_3 = 1$$
$$-4\alpha_1 + 9\alpha_2 + 11\alpha_3 = 1$$

Using symmetry:

$$\alpha_2 = \alpha_3 = \alpha$$

$$-2\alpha_1 + 8\alpha = -1 \Rightarrow \alpha_1 = 4\alpha + \frac{1}{2}$$

Solution for α_i

Substitute:

$$-4(4\alpha + 1/2) + 20\alpha = 1$$

$$-16\alpha - 2 + 20\alpha = 1 \Rightarrow 4\alpha = 3 \Rightarrow \alpha = 0.75$$

$$\alpha_1 = 3.5, \quad \alpha_2 = 0.75, \quad \alpha_3 = 0.75$$

Compute w'

$$w' = \sum \alpha_i y_i \bar{S}_i$$

$$w' = (3.5)(-1)(1, 0, 1) + (0.75)(1)(3, 1, 1) + (0.75)(1)(3, -1, 1)$$

$$= (-3.5, 0, -3.5) + (2.25, 0.75, 0.75) + (2.25, -0.75, 0.75)$$

$$= (1, 0, -2)$$

Final Hyperplane

Separate parameters:

$$w = (1, 0), \quad b = -2$$

Decision function:

$$f(x) = w^T x + b = x_1 - 2$$

Final Boundary

$$x_1 = 2$$

Geometric Interpretation

- Hyperplane: $x_1 = 2$
- Vertical line in 2D
- Splits data into:
  - $x_1 > 2 \rightarrow +1$
  - $x_1 < 2 \rightarrow -1$

Key Insight

Hyperplane is a line (not a point)

In-Class Practice: Find the Optimal SVM Hyperplane

Problem

Consider the following labeled data points:

$$(1, 0.5),\quad (1, 1),\quad (1, -0.5),\quad (-0.5, 0.5),\quad (0.5, 0.5)$$
$$(2, 0),\quad (4, 0),\quad (4.5, 1),\quad (4.5, 0.5),\quad (5, -1),\quad (5.5, 0)$$

Assume the first group belongs to class -1 and the second group belongs to class $+1$.

Your Challenge

Find the **optimal maximum-margin hyperplane**.

Work in pairs — 10 minutes

Hinge Loss: Piecewise Form

Definition

$$\ell_i = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 1 \\ 1 - y_i f(x_i) & \text{otherwise} \end{cases}$$

Meaning

- No penalty if correctly classified with margin
- Linear penalty inside margin

SVM Objective Function

Final Loss

$$J(w, b) = \lambda \|w\|^2 + \sum_{i=1}^{n} \max\left(0,\ 1 - y_i(w^T x_i + b)\right)$$

Components

- $\lambda \|w\|^2 \rightarrow$ Maximizes margin
- Hinge loss $\rightarrow$ Penalizes errors

Goal

Minimize total loss

Per-Sample Contribution

If Margin Satisfied

$$J_i = \lambda \|w\|^2$$

If Margin Violated

$$J_i = \lambda \|w\|^2 + (1 - y_i(w^T x_i + b))$$

Insight

Only violating points add extra loss

Gradient: Case 1 (Correct Classification)

Condition

$$y_i(w^T x_i + b) \geq 1$$

Gradients

$$\frac{\partial J_i}{\partial w} = 2\lambda w$$

$$\frac{\partial J_i}{\partial b} = 0$$

Interpretation

Only regularization affects update

Gradient: Case 2 (Violation)

Condition

$$y_i(w^T x_i + b) < 1$$

Gradients

$$\frac{\partial J_i}{\partial w} = 2\lambda w - y_i x_i$$

$$\frac{\partial J_i}{\partial b} = -y_i$$

Interpretation: Misclassified points push decision boundary.

Key Insight

What SVM Does

- Maximize margin (via $\|w\|^2$)
- Penalize violations (via hinge loss)

Final Intuition

- Only points inside margin matter
- Others are ignored (zero loss)

SVM focuses only on "difficult" points

SVM Update Rule

Case 1: Correct Classification

$$y_i(w^T x_i + b) \geq 1$$

Update

$$w = w - \alpha(2\lambda w)$$

$$b = b$$

Interpretation

Only regularization updates weights

SVM Update Rule

Case 2: Margin Violation

$$y_i(w^T x_i + b) < 1$$

Update

$$w = w - \alpha(2\lambda w - y_i x_i)$$

$$b = b + \alpha y_i$$

Interpretation

Misclassified points push decision boundary

SVM Training Algorithm

Steps

1. Initialize $w = 0,\ b = 0$
2. Ensure labels $y \in \{-1, +1\}$
3. Repeat for multiple iterations:
   - Pick a data point (x_i, y_i)
   - Apply update rule

Type

Stochastic Gradient Descent (SGD)

Prediction

Decision Function

$$f(x) = w^T x + b$$

Prediction Rule

$$\hat{y} = \text{sign}(w^T x + b)$$

Output

- $+1 \rightarrow$ Positive class
- $-1 \rightarrow$ Negative class

SVM Code Structure

```
import numpy as np

class SVM:
    def __init__(self, learning_rate=0.01,
                 lambda_param=0.01,
                 n_iters=1000):

        self.lr = learning_rate
        self.lambda_param = lambda_param
        self.n_iters = n_iters
        self.w = None
        self.b = None
```

Initialization in Training

```
def fit(self, X, y):
    n_samples, n_features = X.shape

    self.w = np.zeros(n_features)
    self.b = 0

    y_ = np.where(y <= 0, -1, 1)
```

Important

Convert labels to $\{-1, +1\}$

Training Loop (SGD)

```
for _ in range(self.n_iters):
    for idx, x_i in enumerate(X):

        condition = y_[idx] * (np.dot(x_i, self.w) + self.b) >= 1
```

Condition

Check if margin constraint is satisfied

Update Rule: Correct Classification

```
if condition:
    self.w -= self.lr * (2 * self.lambda_param * self.w)
    self.b = self.b
```

Interpretation

Only regularization update

Update Rule: Margin Violation

```
else:
    self.w -= self.lr * (2 * self.lambda_param * self.w
                         - y_[idx] * x_i)
    self.b += self.lr * y_[idx]
```

Interpretation

Update driven by misclassified point

Prediction

```
def predict(self, X):
    linear_output = np.dot(X, self.w) + self.b
    return np.sign(linear_output)
```

Output

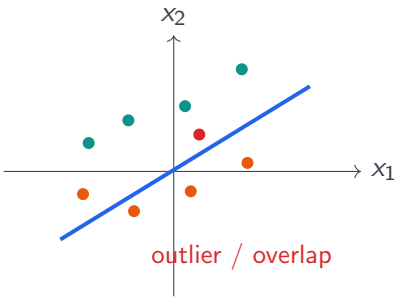
Returns $+1$ or -1

The Problem: Real Data Is Rarely Perfect

Hard-margin SVM assumes

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$$

Every training point must be correctly classified and outside the margin.

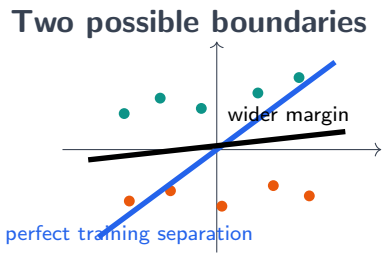


But real data has

- noise and outliers,
- overlapping classes,
- mislabeled examples,
- imperfect measurements.

Key Insight

Why Not Just Separate the Training Data?



The key motivation

Even if training data is linearly separable, a boundary that hugs individual points can generalize poorly.

- A nearby point may be an outlier.
- A label may be noisy.
- A slightly imperfect boundary may have a wider margin.

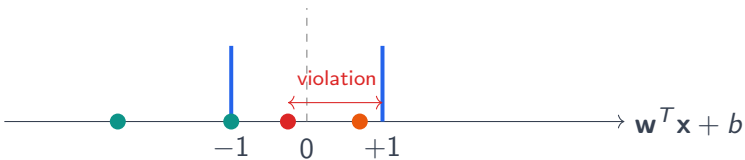
Think

Would you rather memorize every training point — or tolerate a few mistakes for a more robust boundary?

Soft Margin: Relax the Perfect-Separation Rule

Introduce one slack variable ξ_i for every training point.

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i \qquad \xi_i \geq 0$$



- $\xi_i = 0$: margin requirement is satisfied.
- $0 < \xi_i \leq 1$: correct side, but inside the margin.
- $\xi_i > 1$: point crosses the decision boundary and is misclassified.

What Does the Slack Variable Mean?

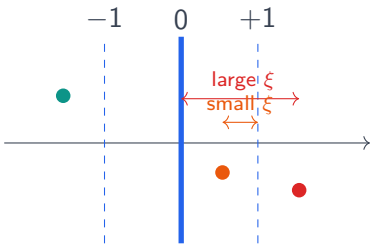
$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$$

$\xi_i = 0 \Rightarrow$ outside/on margin

$0 < \xi_i < 1 \Rightarrow$ inside margin

$\xi_i = 1 \Rightarrow$ on decision boundary

$\xi_i > 1 \Rightarrow$ wrong side



Key Insight

Slack converts a hard constraint into a controlled, measurable violation.

The Soft-Margin Objective

Instead of minimizing only $\frac{1}{2}\|\mathbf{w}\|^2$, we penalize violations:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i$$

subject to

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \qquad \xi_i \geq 0.$$



Key Insight

C controls the trade-off: wide margin versus paying for violations.

What Does C Actually Do?

Small C

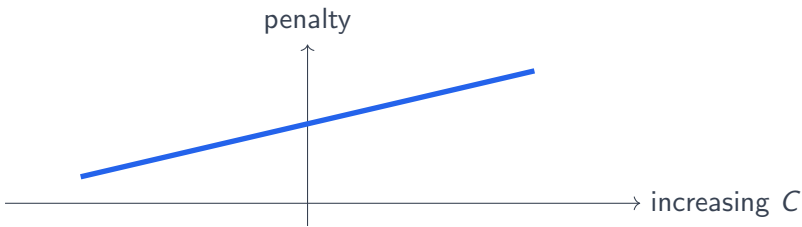
Mistakes are relatively cheap.

- More tolerance for violations.
- More emphasis on a wide margin.
- Boundary can ignore difficult points.

Large C

Mistakes are expensive.

- Strong pressure to classify training points correctly.
- Less tolerance for violations.
- Margin may become narrower.



Soft Margin Through Hinge Loss

The margin is

$$m_i = y_i(\mathbf{w}^T\mathbf{x}_i + b).$$

The smallest feasible slack is

$$\xi_i = \max(0, 1 - m_i).$$

Therefore, the soft-margin objective becomes

$$\frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \max\left(0, 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b)\right)$$

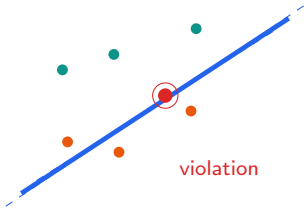


$$L_{\text{hinge}} = \max(0, 1 - m_i)$$

Soft-Margin Support Vectors

In hard-margin SVM, support vectors lie on the margin. With soft margin, influential points can include:

- points exactly on the margin;
- points inside the margin;
- misclassified points.



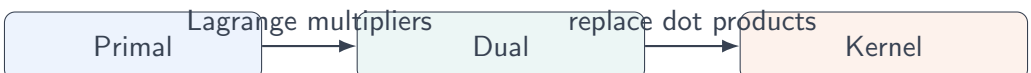
Key Insight

Soft-margin support vectors are not restricted to the two margin hyperplanes.

The Bridge to the Kernel Trick

The dual SVM objective contains pairwise dot products:

$$W(\alpha) = \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j (\mathbf{x}_i^T \mathbf{x}_j).$$



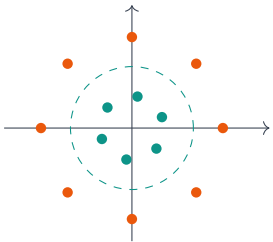
Key Insight

Because the dual only needs pairwise dot products, we can replace them with kernel evaluations.

Why Do We Need the Kernel Trick?

Problem

Some class patterns are nonlinear in the original feature space.



Idea

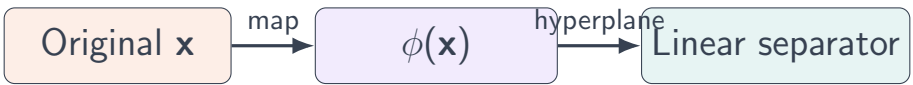
Transform the data:

$$\mathbf{x} \longrightarrow \phi(\mathbf{x})$$

and find a linear separator in the transformed space.

$$K(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle$$

Kernel Trick: The Big Idea



$$K(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle$$

The trick

If the dual only needs dot products, replace

$$\mathbf{x}_i^T \mathbf{x}_j \quad \longrightarrow \quad K(\mathbf{x}_i, \mathbf{x}_j).$$

Polynomial Kernel: A Concrete Example

Take

$$P_1 = (x_1, y_1), \qquad P_2 = (x_2, y_2)$$

and

$$\phi(x, y) = (x^2, y^2, \sqrt{2}xy).$$

Then

$$\begin{aligned} K(P_1, P_2) &= \langle \phi(P_1), \phi(P_2) \rangle \\ &= x_1^2 x_2^2 + y_1^2 y_2^2 + 2x_1 y_1 x_2 y_2 \\ &= \boxed{(x_1 x_2 + y_1 y_2)^2} \\ &= \boxed{(P_1^T P_2)^2}. \end{aligned}$$

A 3-D dot product is computed using only a squared dot product in the original 2-D space.

Why the Kernel Trick Is "Cheap"

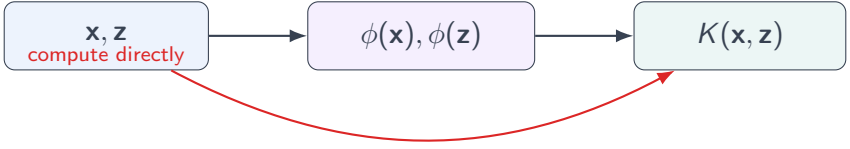
Explicit mapping

Compute $\phi(\mathbf{x})$, $\phi(\mathbf{z})$, then their dot product.

Kernel approach

Compute directly:

$$K(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle.$$



Key Insight

The classifier behaves as if it works in a richer feature space without explicitly constructing that space.

Common Kernel Choices

Kernel	Formula	Typical role
Linear	$K(x,z) = x^T z$	Already linearly separable
Polynomial	$K(x,z) = (x^T z + c)^d$	Polynomial interactions
RBF / Gaussian	$K(x,z) = e^{-\gamma\|x-z\|^2}$	Flexible nonlinear boundaries
Sigmoid	$K(x,z) = \tanh(\gamma x^T z + c)$	Neural-style similarity

Practical choice

The source emphasizes choosing an appropriate kernel using domain knowledge and cross-validation.

Soft Margin vs. Kernel Trick

Soft Margin answers

"What if the classes overlap?"

- Keep a linear boundary.
- Permit controlled violations.
- Penalize violations with $C \sum_i \xi_i$.

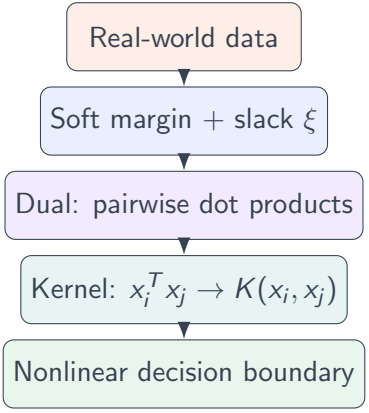
Kernel Trick answers

"What if the boundary is nonlinear?"

- Transform the feature space implicitly.
- Keep the dual formulation.
- Replace dot products with $K(x_i, x_j)$.

Two different problems $\Rightarrow$ two complementary ideas

The Complete SVM Picture



Key Insight

Soft margin handles imperfect separation; kernels handle nonlinear geometry. They can

Take-Home Messages

1. Hard-margin SVM is too rigid for noisy or overlapping data.
2. Slack variables ξ_i quantify margin violations.
3. C controls the trade-off between a wide margin and violations.
4. Soft-margin support vectors can include points inside the margin and misclassified points.
5. The SVM dual depends on pairwise dot products.
6. The kernel trick replaces those dot products with similarity functions.
7. A nonlinear problem in the original space can become a linear problem in a transformed space.

Relax the constraints when data is messy; change the feature space when geometry is nonlinear.